# RNN Time Series Forecasting Using PyTorch

## 1. Task and Objective

The objective of this exercise was to develop a simple recurrent neural network that forecasts values in a time series. A synthetic sine wave with added random noise was generated, and the program learned to predict the next value from a fixed window of previous values. The work demonstrates data generation, sequence preparation, RNN training, loss monitoring, and prediction visualization.

GitHub Link: The local repository is initialized and ready to publish as a separate GitHub repository from Visual Studio Code.

## 2. Approach Used

A reproducible noisy sine series was generated with NumPy. The program creates one example for each target point: the 30 immediately preceding observations form the input, and the next observation is the target. This produces 970 input-output examples from 1,000 time points. The split is chronological: 770 examples for training and 200 for evaluation. For each holdout point, the input window contains the preceding observed values, so the experiment measures one-step-ahead prediction rather than repeatedly feeding predictions back into the model.

## 3. Technologies and Tools

Python was used as the programming language. PyTorch provides the recurrent and linear layers, MSE loss, and Adam optimizer. NumPy creates the synthetic series and sequence windows, while Matplotlib creates the figures. The project is organized for opening and running in Visual Studio Code with a Python virtual environment.

## 4. Synthetic Time Series Data

The program generated 1,000 evenly spaced points from 0 to 40 pi. Each clean value is calculated as sin(t). NumPy then adds independent Gaussian noise with mean 0 and standard deviation 0.12, creating realistic variation around the smooth periodic signal. The random seed is fixed at 42 so that both the series and training initialization can be reproduced. The data plot is saved as results/sine_data.png, and the generated time and noisy values are saved in results/sine_data.csv. The first 800 points define the training portion; the last 200 points are held out.

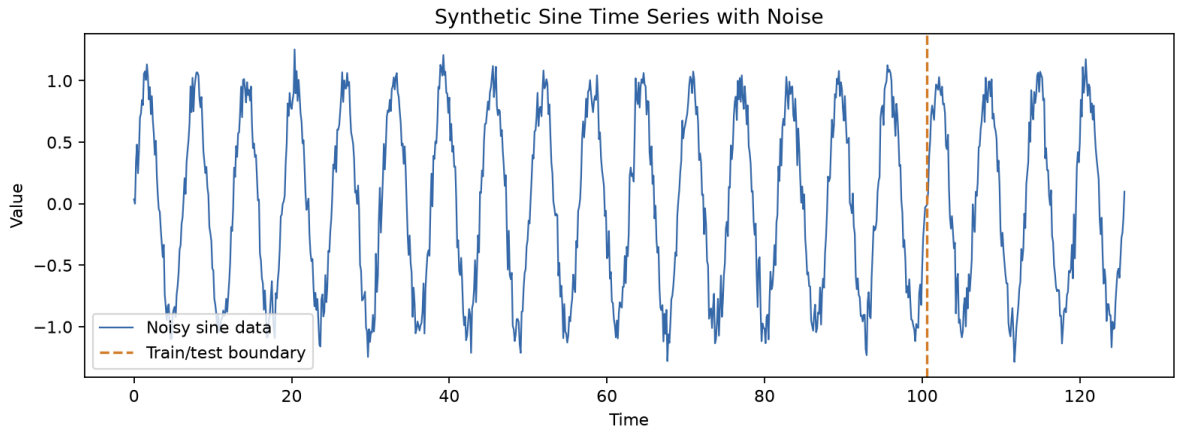



*Figure 1. Generated sine time series with added noise and the chronological train/test boundary.*

## 5. Implementation

The implementation is contained in main.py. The make_data() function generates the signal, then uses a sliding window to create the examples. Each input tensor has shape (30, 1): 30 time steps and one measured value per step. Each target has shape (1), the next value in the series. The SineRNN class applies nn.RNN(input_size=1, hidden_size=32, batch_first=True). It passes the final recurrent output, a 32-value representation of the sequence, to nn.Linear(32, 1) for the prediction. The model has 1,153 trainable parameters. The program trains in shuffled batches of 64, uses Mean Squared Error and Adam with a learning rate of 0.003, and runs for 50 epochs. The chronological holdout remains separate from the optimizer updates.

## 6. Working Procedure

1. Generate a reproducible sine signal and add Gaussian noise.

2. Plot and save the generated series.

3. Create 30-step input windows paired with their next values.

4. Train the RNN on the first 770 windows using MSE loss and Adam.

5. Print training loss at regular intervals.

6. Predict the 200 values in the chronological holdout section.

7. Calculate held-out MSE, save the predictions and model, and plot the result.

## 7. RNN Model

The network contains one recurrent layer and one prediction layer. PyTorch’s default RNN activation is tanh. At each time step, the recurrent layer updates its hidden state using the current noisy value and the prior hidden state. After it processes all 30 inputs, the last output summarizes the window. The linear layer maps those 32 hidden values to a single next-step estimate. The model is intentionally small and directly uses the nn.RNN and nn.Linear layers requested in the exercise.

## 8. Execution and Result

The program was run successfully in the project virtual environment on CPU. It processed 770 training windows and 200 chronological holdout windows. The reported training MSE fell from 0.413535 in epoch 1 to about 0.0193 by epoch 10, then remained near 0.02 through epoch 50. The final logged training MSE was 0.020909. On the 200 holdout examples, the average squared prediction error was 0.019191. The terminal screenshot below shows the command and the completed run output.

```
Device: CPU | Training samples: 770 | Test samples: 200
Input window: 30 time steps | Epochs: 50
Epoch 01/50 - training MSE: 0.413535
Epoch 10/50 - training MSE: 0.019286
Epoch 20/50 - training MSE: 0.019292
Epoch 30/50 - training MSE: 0.020323
Epoch 40/50 - training MSE: 0.019426
Epoch 50/50 - training MSE: 0.020909
Held-out one-step test MSE: 0.019191
Saved plots, data, predictions, and model.
```

```
PS C:\Users\cupsa\Documents\Codex\2026-10-28\referenced-chatgpt-conversation-this-is-an-
NN-Time-Series> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\cupsa\Document
s\Codex\2026-10-28\referenced-chatgpt-conversation-this-is-an-3\outputs\Tutorial-11-RNN-Time-Series\.venv\Scrip
ts\Activate.ps1)
(.venv) PS C:\Users\cupsa\Documents\Codex\2026-10-28\referenced-chatgpt-conversation-this-is-an-3\outputs\Tutor
ial-11-RNN-Time-Series> python main.py
Device: CPU | Training samples: 770 | Test samples: 200
Input window: 30 time steps | Epochs: 50
Epoch 01/50 - training MSE: 0.413535
Epoch 10/50 - training MSE: 0.019286
Epoch 20/50 - training MSE: 0.019292
Epoch 30/50 - training MSE: 0.020323
Epoch 40/50 - training MSE: 0.019426
Epoch 50/50 - training MSE: 0.020909
Held-out one-step test MSE: 0.019191
Saved plots, data, predictions, and model to: C:\Users\cupsa\Documents\Codex\2026-10-28\referenced-chatgpt-conv
ersation-this-is-an-3\outputs\Tutorial-11-RNN-Time-Series\results
(.venv) PS C:\Users\cupsa\Documents\Codex\2026-10-28\referenced-chatgpt-conversation-this-is-an-3\outputs\Tutor
ial-11-RNN-Time-Series>
```

*Figure 2. VS Code terminal screenshot showing the completed program run and held-out MSE.*

## 9. Prediction Visualization

The prediction plot compares the full noisy series with actual observations and RNN predictions in the held-out interval. The orange predictions follow the repeated peaks and troughs in the blue actual values. They are smoother than individual observations because the target series includes random noise. The dashed vertical line marks the point at which the holdout begins. These are one-step predictions: each window uses the preceding observed 30 values.

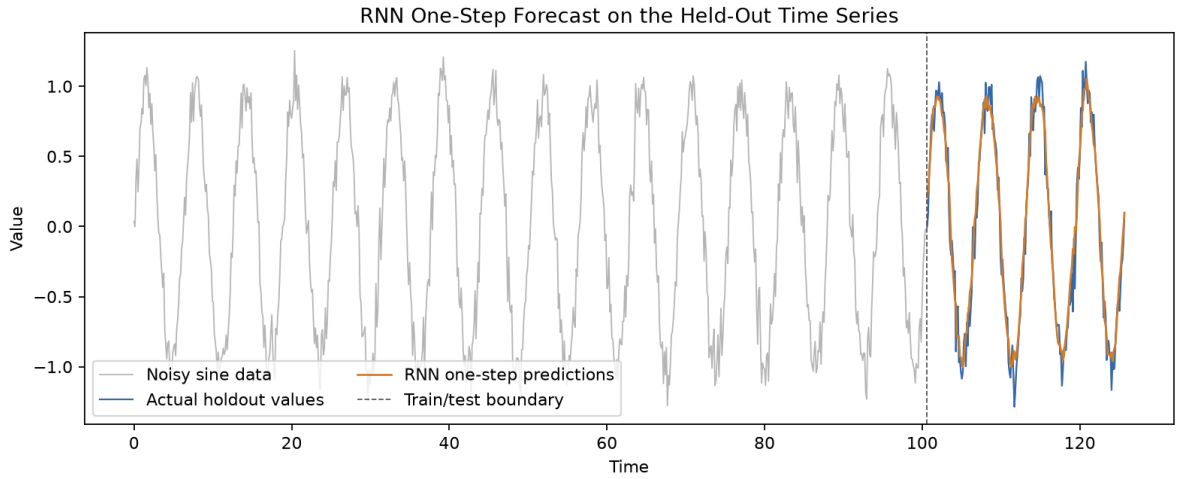



*Figure 3. Actual and predicted values in the held-out portion of the sine time series.*

## 10. Result and Observation

Mean Squared Error is the average of the squared differences between predicted and actual values; lower values indicate closer predictions. The held-out MSE of 0.019191 is small relative to the signal range of roughly -1 to 1, and the plot confirms that the model learned the repeating pattern. Some differences remain around noisy peaks and troughs. Since each test input uses actual prior observations, this result should not be interpreted as a 200-step forecast generated without new observations. The synthetic experiment also does not establish performance on real-world series.

## 11. Conclusion

A PyTorch RNN time-series forecasting program was implemented and executed successfully. It generates noisy sine data, constructs fixed-window sequences, trains an nn.RNN and nn.Linear model with MSE loss and Adam, prints training loss, and visualizes the predictions. The experiment demonstrates the basic RNN forecasting workflow and produces repeatable results with a fixed random seed.